VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

CareerOrbit: Student Placement Management and Recruitment Database System

A9512 - Data Base Management System Laboratory

Submitted by

M. Pranitha	Roll No - 25881A0535
M. Siri Chandana	Roll No - 25581A0555
B. Sanjana	Roll No - 25881A0559

Course Facilitator

Ms. Shaik Shahanaz
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Autonomous Institute, Affiliated to JNTUH,
Approved by AICTE, Accredited by NAAC with A++ Grade
Kacharam, Shamshabad, Hyderabad-501 218,
Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"CareerOrbit: Student Placement Management and Recruitment Database System"** is carried out by **"M. Pranitha", "M. Siri Chandana" and "B. Sanjana"** with Roll Numbers **"25881A0535", "25881A0555" and "25881A0559"** towards **A9512 – Database Management System Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology in Computer Science and Engineering** during the Academic year 2026–27.

Course Facilitator	Academic Head	Head of the Department
Ms. Shaik Shahanaz	**Dr. R. Karthikeyan**	**Prof. Y. Vijayalata**
Assistant Professor, CSE	Professor, CSE	Department of CSE

Contents

1 Introduction 4
2 Problem Analysis 5
2.1 Problem Statement 5
2.2 Users and Assumptions 5
2.3 Functional Requirements 5
3 ER Model 6
3.1 Entities and Attributes 6
3.2 Relationships 6
3.3 Cardinality and Participation 6
3.4 ER Diagram 7
4 Relational Model 8
4.1 Relational Schema 8
4.2 Primary Keys and Foreign Keys 8
5 Normalization 9
5.1 Functional Dependencies 9
5.2 First Normal Form (1NF) 9
5.3 Second Normal Form (2NF) 9
5.4 Third Normal Form (3NF) 10
6 SQL DDL Commands 10
6.1 Table Creation 10
6.2 Constraints 11
6.3 ALTER and DROP Operations 12
7 SQL DML Commands 12
7.1 INSERT 12
7.2 UPDATE 12
7.3 DELETE 13
8 SQL Queries 13
8.1 Basic SELECT Queries 13
8.2 Joins 14
8.3 Aggregate Functions 14
8.4 GROUP BY and HAVING 15
8.5 Subqueries 15
8.6 Views 16
9 Transactions and Concurrency 16
9.1 Transaction Management 16
9.2 COMMIT, ROLLBACK and SAVEPOINT 16

9.3 Concurrency Control . . . 17
10 PL/SQL Implementation . . . 17
10.1 Functions . . . 17
10.2 Stored Procedures . . . 17
10.3 Cursors . . . 17
10.4 Triggers . . . 18
11 File Organization and Indexing . . . 18
11.1 File Organization . . . 18
11.2 ISAM . . . 18
11.3 B+ Tree Indexing . . . 19
12 Testing . . . 19
12.1 Test Cases . . . 19
12.2 Normal and Boundary Cases . . . 19
12.3 Invalid Input Testing . . . 19
13 Reports . . . 20
13.1 Student Eligibility Report . . . 20
13.2 Application and Interview Report . . . 20
13.3 Placement and Recruitment Report . . . 20
14 Sustainable Development Goals (SDGs) and Program Outcomes (POs) Mapping 20
15 Results and Discussion . . . 21
16 Conclusion . . . 21
17 Bibliography . . . 22

Abstract

The **Student Placement System** is a database-driven application designed to streamline, automate, and centralize the recruitment and campus placement drive process in educational institutions. Traditional placement management systems rely heavily on spreadsheets and manual verification, leading to data redundancy, eligibility calculation errors, delayed updates, and communication gaps. The proposed system provides a centralized platform for managing students, companies, job openings, applications, interviews, and placement offers.

The database supports essential operations such as posting job openings, filtering eligible students based on academic criteria such as CGPA, backlogs, and branch, scheduling interview rounds, tracking student application statuses, and maintaining official placement records. The project demonstrates practical DBMS concepts including ER modeling, relational schema normalization up to 3NF, SQL DDL/DML, advanced queries with joins and subqueries, transaction control, concurrency handling, PL/SQL stored routines, indexing, and report generation.

Keywords: *Student Placement System, Database Management System, SQL, Eligibility Filtering, Job Postings, PL/SQL, Triggers, B+ Tree Indexing .*

1 Introduction

CareerOrbit: Student Placement Management and Recruitment Database System

CareerOrbit is a database-driven application designed to simplify, organize, and manage the student placement and recruitment process in colleges and universities. Campus placements involve handling large amounts of information related to students, companies, job opportunities, eligibility criteria, applications, interview schedules, and final selection results. Managing these activities manually can be time-consuming, difficult to maintain, and prone to errors such as duplicate records, data inconsistency, and delays in retrieving important information. CareerOrbit aims to overcome these challenges by providing a centralized and structured database system for efficient placement management.

The proposed system maintains student records, including personal details, academic qualifications, skills, branch, semester, and academic performance. It also stores company information, job descriptions, salary packages, required qualifications, minimum academic criteria, and other recruitment requirements. Based on these details, the system helps placement coordinators identify eligible students for specific job opportunities. This makes the recruitment process more organized and reduces the effort required to manually verify student eligibility.

CareerOrbit also manages the complete placement process, from announcing job opportunities to tracking applications, scheduling interviews, recording selection results, and maintaining final placement records. Students' application statuses can be updated at different stages of recruitment, such as applied, shortlisted, interviewed, selected, or rejected. This allows placement coordinators to monitor recruitment progress and retrieve relevant information through SQL queries. The system can generate useful reports, including lists of eligible candidates, company-wise placement details, student application histories, and final selection statistics.

The project focuses on applying fundamental and advanced Database Management System (DBMS) concepts to a practical real-world problem. It includes the design of a relational database using entities, attributes, primary keys, foreign keys, and relationships. Normalization techniques are applied to reduce data redundancy and improve data consistency. SQL Data Definition Language (DDL) and Data Manipulation Language (DML) commands are used to create tables, define constraints, insert records, and modify or retrieve information. SQL queries involving joins, aggregate functions, subqueries, and filtering conditions are used to extract meaningful information from the database.

Furthermore, the project demonstrates transaction management and concurrency control to maintain data accuracy when multiple operations are performed simultaneously. PL/SQL functions, stored procedures, cursors, and triggers are incorporated to automate database operations, process records, and enforce specific business rules. Concepts related to file organization, Indexed Sequential Access Method (ISAM), and B+ Tree indexing are also explored to understand how database records can be stored, accessed, and retrieved efficiently.

The primary objective of CareerOrbit is to develop a reliable, organized, and efficient placement management solution that reduces manual effort, improves record accuracy, and supports better decision-making during campus recruitment. By integrating different DBMS concepts into a single application, the project demonstrates how database technology can solve practical administrative challenges in educational institutions.

Overall, CareerOrbit provides a structured approach to managing placement information while serving as an effective academic project for understanding database design, SQL programming,

transaction processing, and indexing techniques.

2 Problem Analysis

2.1 Problem Statement

Student placement activities require the management of information related to students, companies, job opportunities, eligibility criteria, applications, interviews, and placement results. When these activities are maintained manually or using separate records, it becomes difficult to identify eligible students, track applications, update interview information, and maintain accurate placement records.

CareerOrbit aims to provide a centralized database system that organizes placement and recruitment information and allows authorized users to efficiently store, update, search, and retrieve the required information.

2.2 Users and Assumptions

The major users of the system are:

- **Placement Coordinator:** Manages student, company, job, application, interview, and placement information.
- **Student:** Views available job opportunities and maintains application-related information.
- **Recruiter:** Provides company and job information and participates in the recruitment process.

The system assumes that student details and academic information are available before the placement process begins. Companies provide job opportunities along with eligibility criteria such as minimum CGPA and eligible departments.

2.3 Functional Requirements

The system provides the following functions:

- Manage student details.
- Manage company details.
- Manage job opportunity details.
- Maintain eligibility criteria.
- Identify eligible students for job opportunities.
- Manage student applications.
- Track application status.
- Maintain interview schedules and results.

- Maintain final placement details.
- Generate placement and recruitment reports.

3 ER Model

3.1 Entities and Attributes

The major entities used in the CareerOrbit system are Student, Company, Job, Application, Interview, and Placement.

- **Student:** Student ID, Student Name, Department, CGPA, Email, Phone.
- **Company:** Company ID, Company Name, Location, Industry.
- **Job:** Job ID, Company ID, Job Role, Package, Minimum CGPA, Eligible Department.
- **Application:** Application ID, Student ID, Job ID, Application Date, Status.
- **Interview:** Interview ID, Application ID, Interview Date, Interview Time, Round, Status.
- **Placement:** Placement ID, Student ID, Company ID, Job ID, Placement Date, Package.

3.2 Relationships

The major relationships between the entities are:

- A company can offer multiple job opportunities.
- A student can apply for multiple job opportunities.
- A job opportunity can receive applications from multiple students.
- An application can have multiple interview rounds.
- A student can receive a placement after successfully completing the recruitment process.
- A company can select multiple students for different job opportunities.

3.3 Cardinality and Participation

The major cardinality relationships in the system are:

- Company to Job: One-to-Many.
- Student to Application: One-to-Many.
- Job to Application: One-to-Many.
- Application to Interview: One-to-Many.
- Student to Placement: One-to-One or One-to-Many depending on the placement rules.
- Company to Placement: One-to-Many.

3.4 ER Diagram

The ER diagram represents the entities, attributes, relationships, cardinality, and participation involved in the CareerOrbit Student Placement Management and Recruitment Database System.

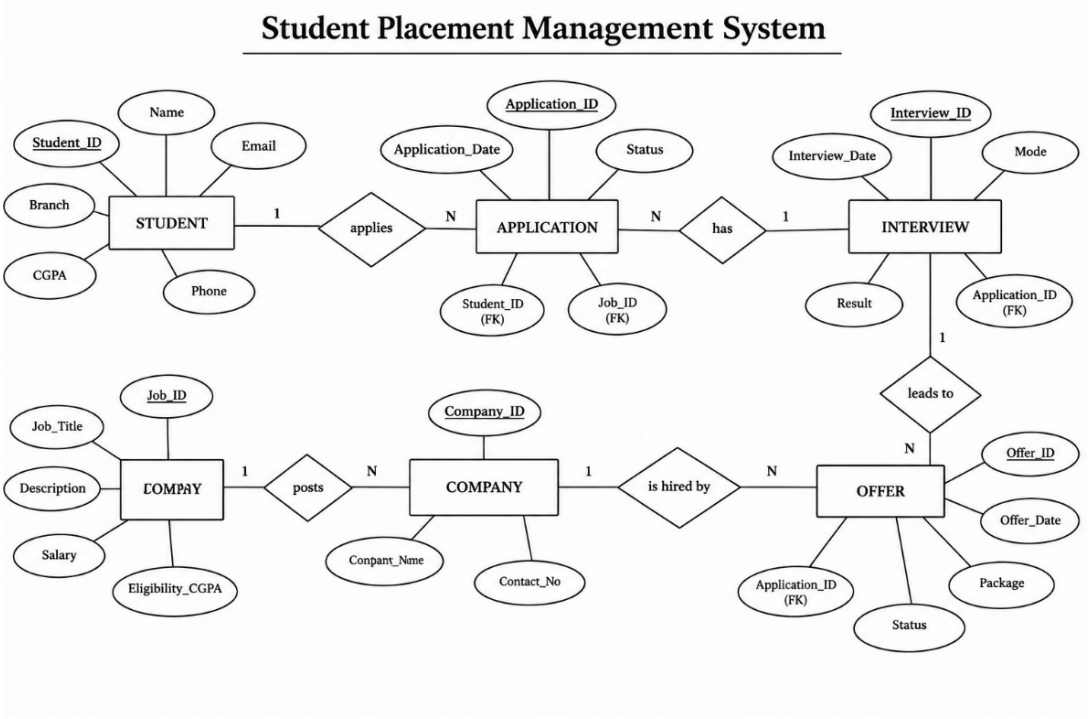


Figure 1: ER Diagram of CareerOrbit Student Placement Management and Recruitment Database System

4 Relational Model

4.1 Relational Schema

The ER model is converted into relational tables using primary keys and foreign keys.

- STUDENT(Student_ID, Student_Name, Department, CGPA, Email, Phone)
- COMPANY(Company_ID, Company_Name, Location, Industry)
- JOB(Job_ID, Company_ID, Job_Role, Package, Min_CGPA, Eligible_Department)
- APPLICATION(Application_ID, Student_ID, Job_ID, Application_Date, Status)
- INTERVIEW(Interview_ID, Application_ID, Interview_Date, Interview_Time, Round, Status)
- PLACEMENT(Placement_ID, Student_ID, Company_ID, Job_ID, Placement_Date, Package)

4.2 Primary Keys and Foreign Keys

Primary keys uniquely identify records in each table, while foreign keys establish relationships between related tables.

- Student_ID is the primary key of STUDENT.
- Company_ID is the primary key of COMPANY.
- Job_ID is the primary key of JOB.
- Application_ID is the primary key of APPLICATION.
- Interview_ID is the primary key of INTERVIEW.
- Placement_ID is the primary key of PLACEMENT.
- Company_ID in JOB references COMPANY.
- Student_ID and Job_ID in APPLICATION reference STUDENT and JOB.
- Application_ID in INTERVIEW references APPLICATION.
- Student_ID, Company_ID, and Job_ID in PLACEMENT reference the corresponding tables.

5 Normalization

5.1 Functional Dependencies

Functional dependencies describe the relationship between attributes in a relation.

The major functional dependencies are:

- Student_ID → Student_Name, Department, CGPA, Email, Phone.
- Company_ID → Company_Name, Location, Industry.
- Job_ID → Company_ID, Job_Role, Package, Min_CGPA, Eligible_Department.
- Application_ID → Student_ID, Job_ID, Application_Date, Status.
- Interview_ID → Application_ID, Interview_Date, Interview_Time, Round, Status.
- Placement_ID → Student_ID, Company_ID, Job_ID, Placement_Date, Package.

5.2 First Normal Form (1NF)

The database satisfies First Normal Form because every attribute contains atomic values and there are no repeating groups. Each record can be uniquely identified using its primary key.

For example, the STUDENT table contains atomic values for each student.

Table 1: Example of First Normal Form (1NF)

Student_ID	**Student_Name**	**Department**	**CGPA**	**Phone**
S101	Rahul	CSE	8.5	9876543210
S102	Priya	ECE	8.2	9876543211
S103	Arjun	CSE	9.0	9876543212

Each field contains only one value, so the relation satisfies 1NF.

5.3 Second Normal Form (2NF)

The database satisfies Second Normal Form because it is in First Normal Form and every non-key attribute depends completely on the primary key of its relation.

For example, application details are stored separately so that student and job information do not depend partially on a combined key.

Table 2: Example of Second Normal Form (2NF)

Application_ID	**Student_ID**	**Job_ID**	**Status**
A101	S101	J101	Selected
A102	S102	J102	Applied
A103	S103	J101	Shortlisted

Here, Application_ID uniquely identifies each application and the non-key attributes depend completely on Application_ID. Therefore, the relation satisfies 2NF.

5.4 Third Normal Form (3NF)

The database satisfies Third Normal Form because it is in Second Normal Form and non-key attributes do not depend on other non-key attributes. This reduces data redundancy and improves data consistency.

For example, company details are maintained separately from the JOB table.

Table 3: Example of Third Normal Form (3NF)

Company_ID	Company_Name	Industry
C101	TCS	IT Services
C102	Infosys	IT Services
C103	Microsoft	Technology

Here, Company_Name and Industry are stored in the COMPANY table and Company_ID is used in the JOB table as a foreign key. Thus, non-key attributes do not depend on other non-key attributes, satisfying 3NF.

6 SQL DDL Commands

6.1 Table Creation

Data Definition Language commands are used to create the required tables for the CareerOrbit database. The CREATE TABLE command is used to create tables such as STUDENT, COMPANY, JOB, APPLICATION, INTERVIEW, and PLACEMENT.

```
SQL> CREATE TABLE STUDENT (
  2      Student_ID VARCHAR(10) PRIMARY KEY,
  3      Student_Name VARCHAR(50) NOT NULL,
  4      Department VARCHAR(20) NOT NULL,
  5      CGPA DECIMAL(3,2) CHECK (CGPA >= 0 AND CGPA <= 10),
  6      Email VARCHAR(50) UNIQUE,
  7      Phone VARCHAR(15)
  8  );

Table created.
```

Figure 2: CREATE TABLE command for STUDENT

```
SQL> CREATE TABLE COMPANY (
  2      Company_ID VARCHAR(10) PRIMARY KEY,
  3      Company_Name VARCHAR(50) NOT NULL,
  4      Location VARCHAR(50) NOT NULL,
  5      Industry VARCHAR(30)
  6  );

Table created.
```

Figure 3: CREATE TABLE command for COMPANY

```
SQL> CREATE TABLE JOB (
  2      Job_ID VARCHAR(10) PRIMARY KEY,
  3      Company_ID VARCHAR(10) NOT NULL,
  4      Job_Role VARCHAR(50) NOT NULL,
  5      Package DECIMAL(10,2) NOT NULL,
  6      Min_CGPA DECIMAL(3,2) NOT NULL,
  7      Eligible_Department VARCHAR(30) NOT NULL,
  8      FOREIGN KEY (Company_ID) REFERENCES COMPANY(Company_ID)
  9  );

Table created.
```

Figure 4: CREATE TABLE command for JOB

```
SQL> CREATE TABLE APPLICATION (
  2      Application_ID VARCHAR(10) PRIMARY KEY,
  3      Student_ID VARCHAR(10),
  4      Job_ID VARCHAR(10),
  5      Application_Date DATE NOT NULL,
  6      Status VARCHAR(20) NOT NULL,
  7      FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  8      FOREIGN KEY (Job_ID) REFERENCES JOB(Job_ID)
  9  );

Table created.
```

Figure 5: CREATE TABLE command for APPLICATION

```
SQL> CREATE TABLE INTERVIEW (
  2      Interview_ID VARCHAR(10) PRIMARY KEY,
  3      Application_ID VARCHAR(10),
  4      Interview_Date DATE NOT NULL,
  5      Interview_Time VARCHAR(10) NOT NULL,
  6      Round VARCHAR(10) NOT NULL,
  7      Status VARCHAR(20) NOT NULL,
  8      FOREIGN KEY (Application_ID) REFERENCES APPLICATION(Application_ID)
  9  );

Table created.
```

Figure 6: CREATE TABLE command for INTERVIEW

```
SQL> CREATE TABLE PLACEMENT (
  2      Placement_ID VARCHAR(10) PRIMARY KEY,
  3      Student_ID VARCHAR(10),
  4      Company_ID VARCHAR(10),
  5      Job_ID VARCHAR(10),
  6      Placement_Date DATE NOT NULL,
  7      Package DECIMAL(10,2) NOT NULL,
  8      FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  9      FOREIGN KEY (Company_ID) REFERENCES COMPANY(Company_ID),
 10      FOREIGN KEY (Job_ID) REFERENCES JOB(Job_ID)
 11  );

Table created.
```

Figure 7: CREATE TABLE command for PLACEMENT

6.2 Constraints

Primary key, foreign key, NOT NULL, UNIQUE, and CHECK constraints are used to maintain data integrity and enforce valid values in the database.

6.3 ALTER and DROP Operations

The ALTER command is used to modify the structure of an existing table, while the DROP command is used to remove a database object.

```
SQL> ALTER TABLE Student ADD Phone VARCHAR(15);

Table altered.
```

Figure 8: ALTER TABLE command

```
SQL> DROP TABLE Offer;

Table dropped.
```

Figure 9: DROP COMMAND

7 SQL DML Commands

7.1 INSERT

The INSERT command is used to add student, company, job, application, interview, and placement records to the database.

```
SQL> INSERT INTO Company VALUES (101, 'TCS', 'IT Services', 'careers@tcs.com', 'Hyderabad');
1 row created.
SQL> INSERT INTO Company VALUES (102, 'Microsoft', 'Product', 'recruiting@microsoft.com', 'Hyderabad');
1 row created.
```

Figure 10: INSERT COMMAND

7.2 UPDATE

The UPDATE command is used to modify existing information such as student CGPA, job package, application status, or interview status.

```
SQL> UPDATE Application SET Status = 'Selected' WHERE Application_ID = 401;

1 row updated.
```

Figure 11: UPDATE COMMAND

7.3 DELETE

The DELETE command is used to remove unwanted or incorrect records from the database.

```
SQL> DELETE FROM Student WHERE Student_ID = 202;

1 row deleted.
```

Figure 12: DELETE COMMAND

8 SQL Queries

8.1 Basic SELECT Queries

SELECT queries are used to retrieve required information from the CareerOrbit database. Examples include displaying all students, companies, available jobs, and placement records.

```
SQL> SELECT * FROM Student WHERE Branch = 'CSE' AND CGPA >= 8.00;

STUDENT_ID FIRST_NAME
---------- --------------------------------------------------
LAST_NAME
--------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
BRANCH                                             CGPA   BACKLOGS
-------------------------------------------------- ---------- ----------
PHONE
---------------
       201 Rahul
Sharma
rahul@vce.ac.in

STUDENT_ID FIRST_NAME
---------- --------------------------------------------------
LAST_NAME
--------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
BRANCH                                             CGPA   BACKLOGS
-------------------------------------------------- ---------- ----------
PHONE
---------------
CSE                                                 8.5         0
```

Figure 13: SELECT QUERY

8.2 Joins

Joins are used to retrieve related information from multiple tables. For example, a join can be used to display student names along with the companies they applied to and the job roles they selected.

```
SQL> SELECT S.First_Name, S.Last_Name, J.Job_Title, C.Company_Name, A.Status
  2  FROM Application A
  3  JOIN Student S ON A.Student_ID = S.Student_ID
  4  JOIN Job J ON A.Job_ID = J.Job_ID
  5  JOIN Company C ON J.Company_ID = C.Company_ID;

FIRST_NAME
--------------------------------------------------
LAST_NAME
--------------------------------------------------
JOB_TITLE
--------------------------------------------------------------------------------
COMPANY_NAME
--------------------------------------------------------------------------------
STATUS
--------------------
Rahul
Sharma
Software Engineer

FIRST_NAME
--------------------------------------------------
LAST_NAME
--------------------------------------------------
JOB_TITLE
--------------------------------------------------------------------------------
COMPANY_NAME
--------------------------------------------------------------------------------
STATUS
--------------------
Microsoft
Selected


FIRST_NAME
--------------------------------------------------
LAST_NAME
--------------------------------------------------
JOB_TITLE
--------------------------------------------------------------------------------
COMPANY_NAME
--------------------------------------------------------------------------------
STATUS
--------------------
Anjali
Rao
Software Engineer

FIRST_NAME
--------------------------------------------------
LAST_NAME
--------------------------------------------------
JOB_TITLE
--------------------------------------------------------------------------------
COMPANY_NAME
--------------------------------------------------------------------------------
STATUS
--------------------
Microsoft
Shortlisted
```

Figure 14: JOINS

8.3 Aggregate Functions

Aggregate functions such as COUNT(), SUM(), AVG(), MAX(), and MIN() are used to calculate placement statistics such as the number of students placed, average package, highest package, and number of applications.

```
SQL> SELECT First_Name, Branch, CGPA
  2  FROM Student
  3  WHERE CGPA > (SELECT AVG(CGPA) FROM Student);

FIRST_NAME
--------------------------------------------------
BRANCH                                                  CGPA
-------------------------------------------------- ----------
Rahul
CSE                                                      8.5

Anjali
CSE                                                      9.1
```

Figure 15: AGGREGATE FUNCTIONS

8.4 GROUP BY and HAVING

GROUP BY is used to group placement information based on attributes such as department or company. HAVING is used to filter grouped results.

```
SQL> SELECT S.Branch, COUNT(*) AS Placed_Count
  2  FROM Student S
  3  JOIN Application A ON S.Student_ID = A.Student_ID
  4  WHERE A.Status = 'Selected'
  5  GROUP BY S.Branch
  6  HAVING COUNT(*) >= 1;

BRANCH                                             PLACED_COUNT
-------------------------------------------------- ------------
CSE                                                           1
```

Figure 16: GROUP BY

```
SQL> SELECT Status, COUNT(*) AS Total_Offers, AVG(Package_Offered) AS Average_Package
  2  FROM Offer
  3  GROUP BY Status
  4  HAVING COUNT(*) > 1;

no rows selected
```

Figure 17: HAVING

8.5 Subqueries

Subqueries are used to retrieve information based on the result of another query. For example, a subquery can be used to find students whose CGPA is greater than the average CGPA of all students.

```
SQL> SELECT Student_ID, First_Name, Branch, CGPA
  2  FROM Student
  3  WHERE CGPA > (SELECT AVG(CGPA) FROM Student);

STUDENT_ID FIRST_NAME
---------- --------------------------------------------------
BRANCH                                                  CGPA
-------------------------------------------------- ----------
       201 Rahul
CSE                                                      8.5

       203 Anjali
CSE                                                      9.1
```

Figure 18: SUBQUERIES

8.6 Views

Views are virtual tables created from one or more tables. A view can be created to display student application details, eligible students, or placement information.

```
SQL> CREATE VIEW Placed_Students_View AS
  2  SELECT S.Student_ID, S.First_Name, S.Branch, C.Company_Name, O.Package_Offered
  3  FROM Student S
  4  JOIN Application A ON S.Student_ID = A.Student_ID
  5  JOIN Job J ON A.Job_ID = J.Job_ID
  6  JOIN Company C ON J.Company_ID = C.Company_ID
  7  JOIN Offer O ON A.Application_ID = O.Application_ID
  8  WHERE O.Status = 'Accepted';

View created.

SQL> SELECT * FROM Placed_Students_View;

STUDENT_ID FIRST_NAME
---------- --------------------------------------------------
BRANCH
--------------------------------------------------
COMPANY_NAME
--------------------------------------------------------------------------------
PACKAGE_OFFERED
---------------
       201 Rahul
CSE
Microsoft
          14.5
```

Figure 19: VIEWS

9 Transactions and Concurrency

9.1 Transaction Management

A transaction is a sequence of database operations treated as a single logical unit. Transaction management helps maintain consistency and reliability of placement records.

9.2 COMMIT, ROLLBACK and SAVEPOINT

COMMIT permanently saves changes made during a transaction. ROLLBACK is used to undo uncommitted changes, while SAVEPOINT creates a point within a transaction to which changes can be rolled back.

```
SQL> COMMIT;

Commit complete.
```

Figure 20: COMMIT

```
SQL> ROLLBACK TO App_Inserted;

Rollback complete.
```

Figure 21: ROLLBACK

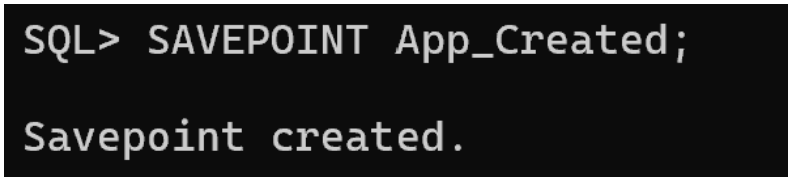


Figure 22: SAVEPOINT

9.3 Concurrency Control

Concurrency control is required when multiple users access or update placement records at the same time.

For example, two placement coordinators may attempt to update the application status of the same student simultaneously. Appropriate transaction and locking mechanisms can be used to prevent inconsistent updates.

10 PL/SQL Implementation

10.1 Functions

PL/SQL functions are used to perform reusable calculations. A function can be used to calculate the average package offered by companies or return placement-related information.



Figure 23: FUNCTIONS

10.2 Stored Procedures

Stored procedures are used to perform database operations. A procedure can be used to register a student application for a particular job opportunity or update the application status.



Figure 24: PROCEDURES

10.3 Cursors

Cursors are used to process multiple rows returned by a query. A cursor can be used to display student placement details or generate a list of students eligible for a particular job.

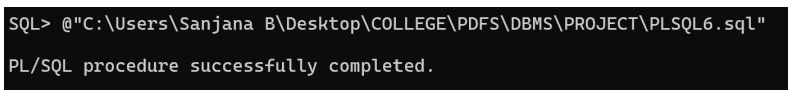


Figure 25: CURSORS

10.4 Triggers

Triggers are automatically executed when specific database operations such as INSERT, UPDATE, or DELETE occur. A trigger can be used to maintain a history of application status changes or automatically perform an action when a placement record is inserted.

```
SQL> @"C:\Users\Sanjana B\Desktop\COLLEGE\PDFS\DBMS\PROJECT\PLSQL4.sql"

Trigger created.
```

Figure 26: TRIGGERS

11 File Organization and Indexing

11.1 File Organization

File organization describes how records are physically stored and accessed in a database system. Efficient file organization helps improve the performance of searching, inserting, updating, and retrieving placement records.

11.2 ISAM

Indexed Sequential Access Method (ISAM) organizes records sequentially while maintaining an index for faster retrieval. It can be used to efficiently search placement records based on attributes such as Student ID or Company ID.

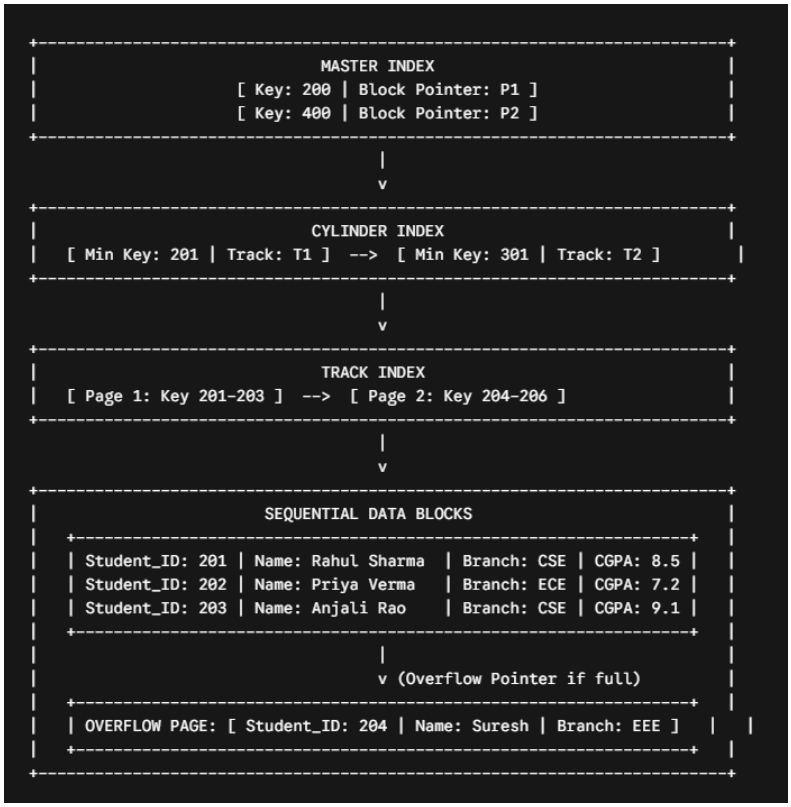


Figure 27: ISAM DIAGRAM

11.3 B+ Tree Indexing

B+ Tree indexing provides efficient searching and retrieval of database records. It organizes search keys in a balanced tree structure and helps reduce the time required to locate records.

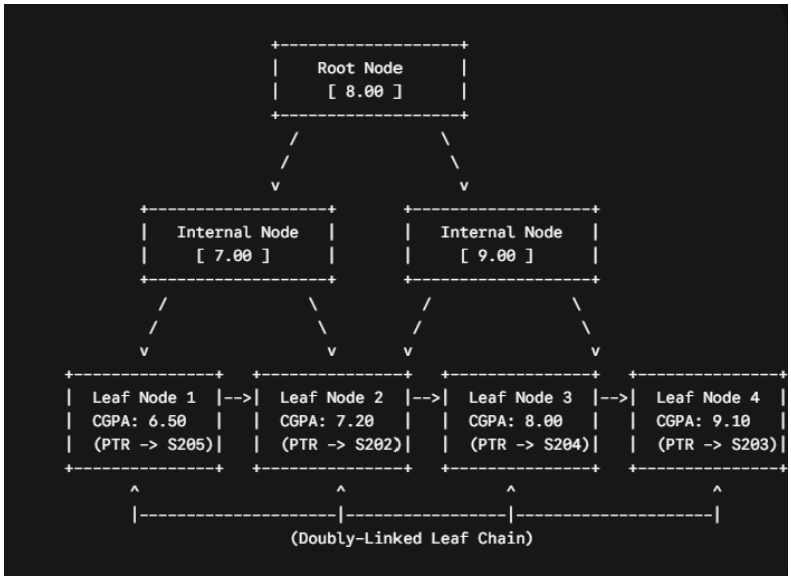


Figure 28: B+ TREE INDEXING DIAGRAM

12 Testing

12.1 Test Cases

Test cases are used to verify that the system performs the expected operations correctly.

- Student registration test.
- Company registration test.
- Job opportunity creation test.
- Student application test.
- Interview scheduling test.
- Placement record test.

12.2 Normal and Boundary Cases

Normal cases verify regular operations such as adding valid student and company records. Boundary cases verify limits such as minimum CGPA requirements, package values, and valid application conditions.

12.3 Invalid Input Testing

Invalid inputs such as incorrect student IDs, non-existing company IDs, invalid job IDs, duplicate applications, and students who do not satisfy eligibility criteria are tested to verify proper error handling.

13 Reports

13.1 Student Eligibility Report

The Student Eligibility Report displays students who satisfy the eligibility criteria for a particular job opportunity based on factors such as department and minimum CGPA.

Table 4: Sample Student Eligibility Report

Student ID	Student Name	Branch	CGPA	Backlogs	Job Role	Company	Status
S201	Rahul Sharma	CSE	8.50	0	Software Engineer	Microsoft	Eligible
S202	Priya Verma	ECE	7.20	1	System Engineer	TCS	Eligible
S203	Anjali Rao	CSE	9.10	0	Software Engineer	Microsoft	Eligible
S204	Suresh Kumar	EEE	6.40	2	Software Engineer	Microsoft	Not Eligible

13.2 Application and Interview Report

The Application and Interview Report displays student details, company details, job role, application status, interview date, interview round, and interview status.

Table 5: Sample Application and Interview Report

App ID	Student Name	Company	Job Role	App Status	Date	Round	Result
A101	Rahul Sharma	Microsoft	Software Engineer	Shortlisted	2026-10-12	Round 1	Passed
A102	Priya Verma	TCS	System Engineer	Shortlisted	2026-10-14	Round 2	Passed
A103	Anjali Rao	Microsoft	Software Engineer	Applied	2026-10-15	Round 1	Pending
A104	Suresh Kumar	TCS	System Engineer	Rejected	2026-10-10	Round 1	Failed

13.3 Placement and Recruitment Report

The Placement and Recruitment Report displays selected students, company names, job roles, placement dates, packages, and recruitment statistics.

Table 6: Sample Placement and Recruitment Report

Student Name	Company Name	Job Role	Placement Date	Package	Status
Rahul Sharma	Microsoft	Software Engineer	2026-10-18	14.50 LPA	Accepted
Priya Verma	TCS	System Engineer	2026-10-19	4.50 LPA	Accepted
Anjali Rao	TCS	System Engineer	2026-10-19	4.50 LPA	Pending

14 Sustainable Development Goals (SDGs) and Program Outcomes (POs) Mapping

To demonstrate alignment with global sustainability standards and academic accreditation criteria, the project's objectives, design methodologies, and outcomes are mapped against relevant United Nations Sustainable Development Goals (SDGs) and standard Program Outcomes (POs) as shown in Table 7.

Table 7: Mapping of Project Objectives with Sustainable Development Goals (SDGs) and Program Outcomes (POs)

Mapping Category	**Target / Identifier**	**Description / Alignment Justification**
Sustainable Development Goals (SDGs)	**SDG 9:** Industry, Innovation, and Infrastructure	The project builds resilient infrastructure, promotes inclusive and sustainable industrialization, and fosters innovation through advanced technological implementation.
	SDG 11: Sustainable Cities and Communities	Contributes to making urban systems and operational frameworks more sustainable, efficient, and resource-optimized.
Program Outcomes (POs)	**PO 3:** Design/Development of Solutions	Design solutions for complex engineering problems and design system components that meet the specified needs with appropriate consideration for public health and safety.
	PO 5: Modern Tool Usage	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools to complex engineering activities.
	PO 11: Project Management and Finance	Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work, as a member and leader in a team.

15 Results and Discussion

The implemented CareerOrbit system successfully demonstrates the practical application of DBMS concepts in student placement and recruitment management. The system organizes student, company, job, application, interview, and placement information in a structured relational database.

The SQL operations provide efficient methods for inserting, updating, deleting, and retrieving placement information. PL/SQL functions, procedures, cursors, and triggers help automate important database operations. Transaction management and concurrency control improve the reliability and consistency of the database.

The generated reports provide useful information about eligible students, applications, interviews, placements, and recruitment activities. Overall, the system provides an organized and efficient solution for managing campus placement activities.

16 Conclusion

CareerOrbit: Student Placement Management and Recruitment Database System provides an organized and efficient database-driven solution for managing student placement and recruitment activities. The system maintains information about students, companies, job opportunities, applications, interviews, and placement records in a structured manner.

The project demonstrates important DBMS concepts including ER modeling, relational modeling, normalization, SQL DDL and DML commands, SQL queries, transactions, concurrency

control, PL/SQL functions, stored procedures, cursors, triggers, file organization, ISAM, and B+ Tree indexing.

By organizing placement information and automating important recruitment operations, the system reduces manual effort, improves data accuracy, and provides an efficient solution for managing student placement activities.

17 Bibliography

1. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*.
2. Silberschatz, A., Korth, H. F. and Sudarshan, S., *Database System Concepts*.
3. Oracle Database Documentation, Oracle Corporation.
4. Course material and laboratory notes provided for Database Management Systems.